



PRACTICAL-01

1. write a program to determine whether the given number is Prime or not?

*what is prime number:- A prime number is a natural number greater than 1 that can only be divided by 1 and itself without leaving a remainder. Examples of prime numbers include 2, 3, 5, 7, 11, 13, 17, 19, 23, and 29.

*why we use prime number:-Prime numbers are the building blocks of number theory. According to the Fundamental Theorem of Arithmetic, every integer greater than 1 can be uniquely factored into prime numbers. This property is crucial for understanding the structure of integers.

*EXPLANATION:-

- **isPrime Function:**

- This function checks if a given number `n` is prime.
- It first eliminates numbers less than or equal to 1.
- It then directly returns true for 2 and 3, as they are prime numbers.
- It eliminates multiples of 2 and 3.
- It then checks for divisibility by all numbers of the form $6k \pm 1$ up to the square root of `n`. This is a more efficient way to check for prime numbers.

- **countPrimes Function:**

- This function counts the number of prime numbers up to a given `limit`.
- It uses the `isPrime` function to check each number from 2 up to `limit` and increments the count for each prime number found.

main Function:

- This function takes an input from the user (the limit up to which prime numbers need to be counted).



- It then calls the `countPrimes` function and prints the number of prime numbers found.

*SUDO CODE:-

function isprime(n)

 if n is 1

 return 0

 if n is 2 or n is 3

 return 1

 if n is divisible by 2 or 3

 return 0

 for i from 5 to sqrt(n) with step 6

 if n is divisible by i or (i + 2)

 return 0

 return 1

main function

 declare n as integer

 print "Enter the number: "

 read n

 ans = isprime(n)

 if ans is 1

 print "prime"



else

 print "not prime"

return 0

*PROGRAM:-

```
#include <stdio.h>
#include <math.h>
int isprime(int n);

int isprime(int n)
{
    if (n==1)
        return 0;
    if(n==2 || n==3)
        return 1;
    if(n%2==0 || n%3==0)
        return 0;
    for(int i=5;n<=sqrt(n);i=i+6)
    {
        if(n%i==0 || n%(i+2)==0)
            return 0;
    }
    return 1;
}

int main()
{
    int num, n;
    printf("Ch.keshavardhan 2203031050137\n");
    printf("Enter the number : ");
    scanf("%d",&n);
    int ans = isprime(n);
    if(ans==1)
        printf("prime");
    else
        printf("not prime");
    return 0;
}
```



PARUL UNIVERSITY
FACULTY ENGINEERING & TECHNOLOGY
B.TECH CSE 3rd YEAR 5th SEMESTER
Design and Analysis of Algorithm
Laboratory (303105219)

}

*OUTPUT: -

```
Ch.keshavardhan 2203031050137
Enter the number : 100032
not prime

=== Code Execution Successful ===
```

```
Ch.keshavardhan 2203031050137
Enter the number : 100013
prime

=== Code Execution Successful ===
```



PRACTICAL-02

2. Given a sorted array and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order?

*what is sorted array:- A sorted array is an array in which the elements are arranged in a specific order. The order can be either ascending or descending:

1. **Ascending Order:** Each element is less than or equal to the next element. For example, in an array [1, 2, 3, 4, 5], each element is followed by a larger or equal element.
2. **Descending Order:** Each element is greater than or equal to the next element. For example, in an array [5, 4, 3, 2, 1], each element is followed by a smaller or equal element.

*EXAMPLES:-

- **Ascending Order:** [3, 7, 11, 15, 20]
- **Descending Order:** [20, 15, 11, 7, 3]

*Common Operation:-

- **Binary Search:** An efficient algorithm for finding an element's position in a sorted array.

*what is target value:- A "target value" is a specific value that you are trying to find, achieve, or use as a reference in various contexts. Here are a few examples of where the concept of a target value might be applied:

In Programming :-

- **Search Algorithms:** When you search for a specific value within a data structure (like an array), that specific value you're searching for is called the target value.



PARUL UNIVERSITY
FACULTY ENGINEERING & TECHNOLOGY
B.TECH CSE 3rd YEAR 5th SEMESTER
Design and Analysis of Algorithm
Laboratory (303105219)

***SUDO CODE:-**

```
function find_index(arr, n, K)
```

```
    start = 0
```

```
    end = n - 1
```

```
    while start <= end
```

```
        mid = (start + end) / 2
```

```
        if arr[mid] == K
```

```
            return mid
```

```
        else if arr[mid] < K
```

```
            start = mid + 1
```

```
        else
```

```
            end = mid - 1
```

```
    return end + 1
```

```
function main
```

```
    arr = [1, 3, 5, 6]
```

```
    n = size of arr
```

```
    K = 7
```



```
result = find_index(arr, n, K)
```

```
print result
```

```
return 0
```

***PROGRAM A:-**

```
#include<stdio.h>
int find_index(int arr[], int n, int K)
{
    int start = 0;
    int end = n - 1;
    while (start <= end) {
        int mid = (start + end) / 2;
        if (arr[mid] == K)
            return mid;
        else if (arr[mid] < K)
            start = mid + 1;
        else
            end = mid - 1;
    }
    return end + 1;
}
int main()
{

    int arr[] = { 1, 3, 5, 6 };
    int n = sizeof(arr) / sizeof(arr[0]);
    int K = 7;
    printf("Ch.keshavardhan 2203031050137\n");
    printf("%d",find_index(arr, n, K));
    return 0;
}
```



OUTPUT:-

```
Ch.keshavardhan 2203031050137
4
```

```
=== Code Execution Successful ===
```

***SUDO CODE:-**

```
function sortedarray(Array, i, j, number)
  if i <= j then
    mid = (i + j) / 2
    if Array[mid] == number then
      return mid
    else if Array[mid] < number then
      return sortedarray(Array, mid + 1, j, number)
    else
      return sortedarray(Array, i, mid - 1, number)
  else
    return -1
```

```
function main
  Array = [1, 2, 3, 4, 5, 6]
  print "Array: [1, 2, 3, 4, 5, 6]"
  print "Which number index do you want to find: "
  read vardhan
```

```
n = length of Array
result = sortedarray(Array, 0, n - 1, vardhan)
if result == -1 then
  print "Number not found in list."
else
  print "Number found at index: ", result
return 0
```




*PROGRAM B:-

```
#include <stdio.h>
```

```
int sortedarray(int Array[], int i, int j, int number) {
    if (i <= j) {
        int mid = (i + j) / 2;
        if (number == Array[mid]) {
            return mid;
        } else if (number > Array[mid]) {
            return sortedarray(Array, mid + 1, j, number);
        } else {
            return sortedarray(Array, i, mid - 1, number);
        }
    } else {
        return -1;
    }
}

int main() {
    int Array[] = {1, 2, 3, 4, 5, 6};
    printf("Ch.keshavardhan 2203031050137\n");
    printf("Array: [1, 2, 3, 4, 5, 6]\n");
    int vardhan;
    printf("Which number index do you want to find: ");
    scanf("%d", &vardhan);
    int result = sortedarray(Array, 0, sizeof(Array) / sizeof(Array[0]) - 1,
vardhan);
    if (result == -1) {
        printf("Number not found in list.\n");
    } else {
        printf("Number found at index: %d\n", result);
    }
    return 0;
}
```



PARUL UNIVERSITY
FACULTY ENGINEERING & TECHNOLOGY
B.TECH CSE 3rd YEAR 5th SEMESTER
Design and Analysis of Algorithm
Laboratory (303105219)

```
Ch.keshavardhan 2203031050137  
Array: [1, 2, 3, 4, 5, 6]  
Which number index do you want to find: 2  
Number found at index: 1
```

*OUTPUT:-

```
=== Code Execution Successful ===
```

```
Ch.keshavardhan 2203031050137  
Array: [1, 2, 3, 4, 5, 6]  
Which number index do you want to find: 8  
Number not found in list.
```

```
=== Code Execution Successful ===
```

LEET CODE:-

class Solution(object):

def searchInsert(self, nums, target):

i = 0

r = len(nums)-1

while i <= r:

mid = (i+r)//2

if nums[mid]<target:

i=mid+1

elif nums[mid]>target:

r=mid-1



PARUL UNIVERSITY
FACULTY ENGINEERING & TECHNOLOGY
B.TECH CSE 3rd YEAR 5th SEMESTER
Design and Analysis of Algorithm
Laboratory (303105219)

```
else:  
    return mid  
return i
```

OUTPUT:-

Target: 5, Output: 2

=== Code Execution Successful ===



PRACTICAL-03

3. There are N children standing in a line with some rating value. You want to distribute a minimum number of candies to these children such that: Each child must have at least one candy. The children with higher ratings will have more candies than their neighbours. You need to write a program to calculate the minimum candies you must give.?

*To solve this problem, we need to ensure two main rules are satisfied:

1. Every child gets at least one candy.
2. A child with a higher rating than their neighbors gets more candies than those neighbors.
To achieve this, we'll use a two-pass algorithm. Here's the step-by-step approach:

***Steps to Solve the Problem**

1. **Initialization:**
 - Create an array candies where each element is initialized to 1. This ensures every child gets at least one candy.
2. **Left to Right Pass:**
 - Iterate through the ratings array from left to right. If the rating of the current child is higher than the previous child, increment the current child's candies by 1 more than the previous child's candies.
3. **Right to Left Pass:**
 - Iterate through the ratings array from right to left. If the rating of the current child is higher than the next child, update the current child's candies to be the maximum of its current candies or 1 more than the next child's candies. This ensures that children with higher ratings have more candies than their right-side neighbors as well.

***EXPLANATION:-**

Initialization:

- $\text{candies} = [1] * n$ initializes each child's candy count to 1.

Left to Right Pass:

For each child from the second to the last (i from 1 to n-1), if the current child's rating is higher than the previous child's rating, we set $\text{candies}[i] = \text{candies}[i - 1] + 1$.



Right to Left Pass:

- For each child from the second last to the first (i from $n-2$ to 0), if the current child's rating is higher than the next child's rating, we set $\text{candies}[i] = \max(\text{candies}[i], \text{candies}[i+1] + 1)$ to ensure both the left and right conditions are met.

*SUDO CODE:-

1. Initialize ratings array

```
ratings = [12, 4, 3, 11, 34, 34, 1, 67]
```

2. Determine the number of children

```
n = length of ratings
```

3. Create and initialize candies array

```
for i from 0 to n-1 do
```

```
    candies[i] = 1
```

```
end for
```

4. Left to Right Pass

```
for i from 1 to n-1 do
```

```
    if ratings[i] > ratings[i-1] then
```

```
        candies[i] = candies[i-1] + 1
```

```
    end if
```

```
end for
```

5. Right to Left Pass

```
for i from n-2 to 0 do
```

```
    if ratings[i] > ratings[i+1] and candies[i] <= candies[i+1] then
```

```
        candies[i] = candies[i+1] + 1
```



end if

end for

6. Calculate total candies

total = 0

for i from 0 to n-1 do

total = total + candies[i]

end for

7. Output the result

print "Ch keshavardhan 2203031050137"

print "Minimum number of candies: ", total

***PROGRAM:-**

```
#include <stdio.h>
```

```
int main() {
```

```
    int ratings[] = {12, 4, 3, 11, 34, 34, 1, 67};
```

```
    int n = sizeof(ratings) / sizeof(ratings[0]);
```

```
    int candies[n];
```

```
    int i, total = 0;
```

```
    for (i = 0; i < n; i++) {
```

```
        candies[i] = 1;
```

```
    }
```

```
    for (i = 1; i < n; i++) {
```

```
        if (ratings[i] > ratings[i-1]) {
```

```
            candies[i] = candies[i-1] + 1;
```



```
}
```

```
}
```

```
for (i = n-2; i >= 0; i--) {  
    if (ratings[i] > ratings[i+1] && candies[i] <= candies[i+1]) {  
        candies[i] = candies[i+1] + 1;  
    }  
}  
for (i = 0; i < n; i++) {  
    total += candies[i];  
}  
printf("Ch keshavardhan 2203031050137\n");  
printf("Minimum number of candies: %d\n", total);  
return 0;  
}
```

***OUTPUT:-**

```
Ch keshavardhan 2203031050137  
Minimum number of candies: 16  
  
=== Code Execution Successful ===
```



LEET CODE:-

```
class Solution(object):  
    def candy(self, ratings):  
        """  
        :type ratings: List[int]  
        :rtype: int  
        """  
        n = len(ratings)  
        if n == 0:  
            return 0  
        candies = [1] * n  
        for i in range(1, n):  
            if ratings[i] > ratings[i-1]:  
                candies[i] = candies[i-1] + 1  
        for i in range(n-2, -1, -1):  
            if ratings[i] > ratings[i+1]:  
                candies[i] = max(candies[i], candies[i+1] + 1)  
        return sum(candies)  
ratings = [1, 0, 2]  
solution = Solution()  
print("Minimum candies required:", solution.candy(ratings))
```




PARUL UNIVERSITY
FACULTY ENGINEERING & TECHNOLOGY
B.TECH CSE 3rd YEAR 5th SEMESTER
Design and Analysis of Algorithm
Laboratory (303105219)

OUTPUT:-

```
Minimum candies required: 5  
  
=== Code Execution Successful ===
```

PRACTICAL-04

4. There is a new barn with N stalls and C cows. The stalls are located on a straight line at positions x_1, x_N ($0 \leq x_i \leq 1,000,000,000$). We want to assign the cows to the stalls, such that the minimum distance between any two of them is as large as possible. What is the largest minimum distance?

* This is a classic problem known as the "Aggressive Cows Problem" or "Cows in Stalls Problem".

Steps to Solve the Problem

1. Sort the Stall Positions:

- First, sort the array of stall positions.

2. Binary Search for the Largest Minimum Distance:

- Use binary search to find the largest minimum distance that can be achieved.
- The search range for the binary search will be between 1 (minimum possible distance) and the difference between the maximum and minimum positions in the sorted list of stalls.

3. Check Feasibility:

- For each midpoint in the binary search, check if it's possible to place all cows such that the minimum distance between any two cows is at least this midpoint.
- This is done by a greedy approach where the first cow is placed in the first stall, and each subsequent cow is placed in the next available stall that is at least the midpoint distance away from the last placed cow.

***EXPLANATION:-**

Sorting the Positions:

- We sort the positions array to facilitate the greedy placement of cows.

Binary Search:



- The largest_minimum_distance function performs a binary search on the possible minimum distances.
- It checks if it is feasible to place all cows with at least mid distance apart using the is_feasible function.
- If it is feasible, it updates the best known distance and searches the upper half of the range. Otherwise, it searches the lower half.

Feasibility Check:

- The is_feasible function places the first cow in the first stall and then attempts to place each subsequent cow in the next stall that is at least distance away from the last placed cow.
- If all cows are placed successfully, it returns true; otherwise, it returns false.

***SUDO CODE:-**

```
function largest_minimum_distance(positions, C):
```

```
    sort(positions)
```

```
    low = 1
```

```
    high = positions[N-1] - positions[0]
```

```
    best = 0
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if is_feasible(positions, C, mid):
```

```
            best = mid
```

```
            low = mid + 1
```

```
        else:
```

```
            high = mid - 1
```

```
    return best
```

```
function is_feasible(positions, C, distance):
```

```
    count = 1
```



```
last_position = positions[0]
for i from 1 to length(positions) - 1:
    if positions[i] - last_position >= distance:
        count += 1
        last_position = positions[i]
    if count == C:
        return true
return false
```

***PROGRAM:-**

```
#include <stdio.h>

int can_place_cows(int stalls[], int n, int C, int min_dist) {
    int count = 1;
    int last_position = stalls[0];
    for (int i = 1; i < n; i++) {
        if (stalls[i] - last_position >= min_dist) {
            count++;
            last_position = stalls[i];
            if (count == C) {
                return 1;
            }
        }
    }
    return 0;
}
```



```
int largest_min_distance(int stalls[], int n, int C) {  
    for (int i = 0; i < n-1; i++) {  
        for (int j = i+1; j < n; j++) {  
            if (stalls[i] > stalls[j]) {  
                int temp = stalls[i];  
                stalls[i] = stalls[j];  
                stalls[j] = temp;  
            }  
        }  
    }  
    int low = 0;  
    int high = stalls[n-1] - stalls[0];  
    int result = 0;  
    while (low <= high) {  
        int mid = (low + high) / 2;  
        if (can_place_cows(stalls, n, C, mid)) {  
            result = mid;  
            low = mid + 1;  
        } else {  
            high = mid - 1;  
        }  
    }  
    return result;  
}
```



PARUL UNIVERSITY
FACULTY ENGINEERING & TECHNOLOGY
B.TECH CSE 3rd YEAR 5th SEMESTER
Design and Analysis of Algorithm
Laboratory (303105219)

```
int main() {  
    int stalls[] = {0, 3, 4, 7, 8, 10};  
    int C = 3;  
    int n = sizeof(stalls) / sizeof(stalls[0]);  
    printf("Ch keshavardhan 2203031050137\n");  
    printf("%d\n", largest_min_distance(stalls, n, C));  
    return 0;  
}
```

***OUTPUT:-**

```
Ch keshavardhan 2203031050137  
4  
  
=== Code Execution Successful ===
```

PRACTICAL-05

5. Given an undirected graph with V vertices and E edges, check whether it contains any cycle or not?

*what is undirected graph:- An **undirected graph** is a type of graph in which the edges between vertices (nodes) do not have a direction. This means that the connection between any two vertices is bidirectional. In other words, if there is an edge between vertices A and B, you can traverse from A to B and also from B to A without any restriction.

***DFS:-** DFS (Depth-First Search) is a fundamental algorithm used in computer science for traversing or searching tree or graph data structures. The algorithm explores as far as possible along each branch before backtracking.

How DFS Works:

1. Starting Point: You start at a given node (the root, in the case of a tree).
2. Visit Node: Mark the node as visited.
3. Explore Neighbors: For each adjacent node (neighbor), recursively visit it if it has not been visited yet.
4. Backtracking: When all neighbors of a node have been explored, the algorithm backtracks to the previous node to continue the exploration of other unexplored branches.

Types of DFS Implementation:

1. Recursive DFS: Uses the system's function call stack for backtracking.
2. Iterative DFS: Uses an explicit stack data structure to emulate the recursive behavior.

***EXPLANATION:-**

1. Graph Structure

The graph is represented using an adjacency matrix, which is a 2D array. Each element $adj[i][j]$ in the matrix indicates whether there is an edge between vertex i and vertex j:

- $adj[i][j] = 1$: There is an edge between vertices i and j.
- $adj[i][j] = 0$: There is no edge between vertices i and j.



The Graph structure contains:

- V: The number of vertices.
- adj: The adjacency matrix.

2. Graph Creation

The function `createGraph(int V)` dynamically allocates memory for the graph and its adjacency matrix. The adjacency matrix is initialized to 0, meaning no edges are present when the graph is created.

3. Adding Edges

The `addEdge` function adds an undirected edge between two vertices by updating the adjacency matrix. Since the graph is undirected, when an edge is added between `v` and `w`, the program sets both `adj[v][w]` and `adj[w][v]` to 1.

4. Cycle Detection

The cycle detection logic is implemented using DFS:

- The `isCycleUtil` function performs DFS on the graph starting from vertex `v`. It uses a `visited[]` array to track visited vertices and a `parent` parameter to check for back edges.
- A cycle is detected if:
 - An adjacent vertex of `v` has been visited and is not the parent of `v`. This indicates a back edge, meaning a cycle exists.

The `isCycle` function checks for cycles starting from all vertices to handle disconnected components of the graph.

5. Main Function

The main function interacts with the user to get the number of vertices (`V`) and edges (`E`). It then constructs the graph by calling `addEdge` for each edge provided by the user. After constructing the graph, the program calls `isCycle` to check if a cycle is present. Based on the result, it prints either "Cycle Detected" or "No Cycle Detected".



6. Memory Management

After checking for cycles, the program frees the dynamically allocated memory for the graph and adjacency matrix to avoid memory leaks.

*SUDO CODE:-

Graph Structure:

- Create a graph structure with:
 - V: Number of vertices.
 - adj: Adjacency matrix (2D array).

Create Graph:

- Function createGraph(V):
 - Allocate memory for the graph.
 - Initialize the adjacency matrix with 0 to represent no edges.

Add Edge:

- Function addEdge(graph, v, w):
 - Set adj[v][w] and adj[w][v] to 1 to represent an undirected edge between vertices v and w.

Cycle Detection Utility:

- Function isCycleUtil(graph, v, visited[], parent):
 - Mark vertex v as visited.
 - For each adjacent vertex:
 - If the vertex is not visited, recursively call the utility function.
 - If the vertex is visited and not the parent, a cycle is detected.



Cycle Detection:

- Function isCycle(graph):
 - Initialize all vertices as not visited.
 - For each vertex, if not visited, call the cycle detection utility.
 - If a cycle is detected in any DFS tree, return true.

Main:

- Read the number of vertices V and edges E.
- Create the graph.
- Add edges between vertices as per input.
- Call isCycle() to check for cycles and print the result.

Cleanup:

- Free the memory allocated for the graph and adjacency matrix.

*PROGRAMM:-

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

struct Graph {
    int V;
    int **adj;
};

struct Graph* createGraph(int V) {
    struct Graph* graph = (struct Graph*) malloc(sizeof(struct Graph));
    graph->V = V;
    graph->adj = (int **)malloc(V * sizeof(int *));
```



```
for (int i = 0; i < V; i++) {  
    graph->adj[i] = (int *)malloc(V * sizeof(int));  
    for (int j = 0; j < V; j++) {  
        graph->adj[i][j] = 0;  
    }  
}  
return graph;  
}  
  
void addEdge(struct Graph* graph, int v, int w) {  
    graph->adj[v][w] = 1;  
    graph->adj[w][v] = 1;  
}  
  
bool isCycleUtil(struct Graph* graph, int v, bool visited[], int parent) {  
    visited[v] = true;  
    for (int i = 0; i < graph->V; i++) {  
        if (graph->adj[v][i]) {  
            if (!visited[i]) {  
                if (isCycleUtil(graph, i, visited, v)) {  
                    return true;  
                }  
            }  
            else if (i != parent) {  
                return true;  
            }  
        }  
    }  
}
```



```
    }  
    }  
}  
return false;  
}  
  
bool isCycle(struct Graph* graph) {  
    bool *visited = (bool *)malloc(graph->V * sizeof(bool));  
    for (int i = 0; i < graph->V; i++) {  
        visited[i] = false;  
    }  
    for (int u = 0; u < graph->V; u++) {  
        if (!visited[u]) {  
            if (isCycleUtil(graph, u, visited, -1)) {  
  
return true;  
            }  
        }  
    }  
    return false;  
}  
  
int main() {  
    int V, E;  
    printf("Enter the number of vertices: ");  
    scanf("%d", &V);
```



```
struct Graph* graph = createGraph(V);
printf("Enter the number of edges: ");
scanf("%d", &E);
printf("Enter the edges (vertex1 vertex2):\n");
for (int i = 0; i < E; i++) {
    int v1, v2;
    scanf("%d %d", &v1, &v2);
    addEdge(graph, v1, v2);
}
if (isCycle(graph)) {
    printf("Cycle Detected\n");
} else {
    printf("No Cycle Detected\n");
}

for (int i = 0; i < V; i++) {
    free(graph->adj[i]);
}
free(graph->adj);
free(graph);

return 0;
}
```



PARUL UNIVERSITY
FACULTY ENGINEERING & TECHNOLOGY
B.TECH CSE 3rd YEAR 5th SEMESTER
Design and Analysis of Algorithm
Laboratory (303105219)

***OUTPUT:-**

CYCLE DETECTED:-

```
Enter the number of vertices: 5
Enter the number of edges: 5
Enter the edges (vertex1 vertex2):
0 1
1 2
2 3
3 4
4 1
Cycle Detected
CH.keshavardhan 2203031050137

Process finished with exit code 0
```

NO CYCLE DETECTED:-

```
Enter the number of vertices: 5
Enter the number of edges: 4
Enter the edges (vertex1 vertex2):
0 1
1 2
2 3
3 4
no Cycle Detected
CH.keshavardhan 2203031050137

Process finished with exit code 0
```

PRACTICAL-06

6. There are n servers numbered from 0 to $n - 1$ connected by undirected server-to-server connections forming a network where $\text{connections}[i] = [a_i, b_i]$ represents a connection between servers a_i and b_i . Any server can reach other servers directly or indirectly through the network. A critical connection is a connection that, if removed, will make some servers unable to reach some other servers. Return all critical connections in the network in any order.?

***what is undirected graph:-** An **undirected graph** is a type of graph in which the edges between vertices (nodes) do not have a direction. This means that the connection between any two vertices is bidirectional. In other words, if there is an edge between vertices A and B, you can traverse from A to B and also from B to A without any restriction.

***DFS:-** DFS (Depth-First Search) is a fundamental algorithm used in computer science for traversing or searching tree or graph data structures. The algorithm explores as far as possible along each branch before backtracking.

How DFS Works:

5. Starting Point: You start at a given node (the root, in the case of a tree).
6. Visit Node: Mark the node as visited.
7. Explore Neighbors: For each adjacent node (neighbor), recursively visit it if it has not been visited yet.
8. Backtracking: When all neighbors of a node have been explored, the algorithm backtracks to the previous node to continue the exploration of other unexplored branches.

Types of DFS Implementation:

3. Recursive DFS: Uses the system's function call stack for backtracking.
4. Iterative DFS: Uses an explicit stack data structure to emulate the recursive behavior.

***EXPLANATION:-**

Graph Representation: We first build an adjacency list to represent the network of servers.

DFS and Backtracking: We perform DFS starting from any unvisited node and track the discovery and low values for each node. We update the low values while backtracking.



Critical Edge Detection: If for any node v , $low[neighbor] > discovery[node]$, it means that removing the edge between node and neighbor would disconnect the graph, so it's a critical connection.

Result: The list result contains all the critical connections.

***SUDO CODE:-**

function findCriticalConnections(n , connections):

 build graph from connections

 initialize discovery and low arrays to -1

 initialize time to 0, result to empty list

 function dfs($node$, $parent$):

 set $discovery[node]$ and $low[node]$ to time

 increment time

 for each neighbor of $node$:

 if neighbor is parent, skip

 if neighbor is unvisited:

 dfs($neighbor$, $node$)

 update $low[node]$

 if $low[neighbor] > discovery[node]$:

 add [$node$, $neighbor$] to result

 else:

 update $low[node]$ with back edge

 for each $node$:

 if $node$ is unvisited:

 dfs($node$, -1)

 return result



***PROGRAMM:-**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_NODES 10000 // Maximum number of nodes
#define MAX_EDGES 10000 // Maximum number of edges
typedef struct {
    int *list[MAX_NODES]; // Adjacency list for each node
    int size[MAX_NODES]; // Size of adjacency list for each node
} Graph;
void addEdge(Graph *graph, int u, int v) {
    graph->list[u][graph->size[u]++] = v;
    graph->list[v][graph->size[v]++] = u;
}
void dfs(Graph *graph, int node, int parent, int *discovery, int *low, int *time, int **result, int *resultSize) {
    discovery[node] = low[node] = (*time)++;
    for (int i = 0; i < graph->size[node]; ++i) {
        int neighbor = graph->list[node][i];
        if (neighbor == parent) continue; // Skip the parent node
        if (discovery[neighbor] == -1) { // If neighbor is unvisited
            dfs(graph, neighbor, node, discovery, low, time, result, resultSize);
            low[node] = (low[node] < low[neighbor]) ? low[node] : low[neighbor];

            if (low[neighbor] > discovery[node]) {
```



```
        result[*resultSize][0] = node;
        result[*resultSize][1] = neighbor;
        (*resultSize)++;
    }
} else {
    low[node] = (low[node] < discovery[neighbor]) ? low[node] : discovery[neighbor];
}
}
}

int** findCriticalConnections(int n, int connections[][2], int connectionsSize, int* returnSize) {
    Graph graph;
    memset(&graph, 0, sizeof(graph));
    for (int i = 0; i < n; ++i) {
        graph.list[i] = (int *)malloc(MAX_EDGES * sizeof(int));
    }
    for (int i = 0; i < connectionsSize; ++i) {
        addEdge(&graph, connections[i][0], connections[i][1]);
    }
    int discovery[MAX_NODES], low[MAX_NODES], time = 0;
    memset(discovery, -1, sizeof(discovery));
    memset(low, -1, sizeof(low));
    int **result = (int **)malloc(connectionsSize * sizeof(int *));
    for (int i = 0; i < connectionsSize; ++i) {
        result[i] = (int *)malloc(2 * sizeof(int));
    }
}
```



```
}  
  
*returnSize = 0;  
for (int i = 0; i < n; ++i) {  
    if (discovery[i] == -1) {  
        dfs(&graph, i, -1, discovery, low, &time, result, returnSize);  
    }  
}  
  
for (int i = 0; i < n; ++i) {  
    free(graph.list[i]);  
}  
  
return result;  
}  
  
int main() {  
    int n = 5;  
  
    int connections[][2] = {{0, 1}, {1, 2}, {2, 0}, {1, 3}, {3, 4}};  
    int connectionsSize = sizeof(connections) / sizeof(connections[0])  
    int returnSize;  
  
    int **criticalConnections = findCriticalConnections(n, connections, connectionsSize,  
&returnSize);  
  
    printf("CH.keshavardhan 2203031050137\n");  
    printf("Critical Connections:\n");  
    for (int i = 0; i < returnSize; ++i) {  
        printf("[%d, %d]\n", criticalConnections[i][0], criticalConnections[i][1]);  
        free(criticalConnections[i]); // Free each result pair  
    }  
}
```



PARUL UNIVERSITY
FACULTY ENGINEERING & TECHNOLOGY
B.TECH CSE 3rd YEAR 5th SEMESTER
Design and Analysis of Algorithm
Laboratory (303105219)

```
}  
    free(criticalConnections);  
    return 0;  
}
```

***OUTPUT:-**

```
CH.keshavardhan 2203031050137  
Critical Connections:  
[3, 4]  
[1, 3]  
  
=== Code Execution Successful ===
```



PRACTICAL-07

6. Given a grid of size $N \times M$ (N is the number of rows and M is the number of columns in the grid) consisting of '0's (Water) and '1's (Land). Find the number of islands.

SOLVING STEPS:-

Given a **grid** of size $N \times M$ where:

- N is the number of rows.
- M is the number of columns.
- The grid consists of:
 - '0' representing **Water**.
 - '1' representing **Land**.

An **island** is a group of connected lands ('1's) that are connected **vertically** or **horizontally** (not diagonally). Two land cells are part of the same island if they are connected. An island can be surrounded by water and is formed by connecting adjacent land cells. You need to **count the total number of distinct islands** in the grid.

***EXPLANATION:-**

Step-by-Step Explanation for the Example Grid:

1. Starting from the first row, first column (0, 0):
 - It's a '1', which means it's a part of an island.
 - Perform DFS to explore all connected land cells.
 - Mark (0, 0), (0, 1), (1, 0), and (1, 1) as visited (convert them to '0').
 - Increment the island count to 1.
2. Move to the next unvisited cell, which is (2, 2):
 - It's a '1', indicating the start of a new island.
 - Perform DFS from (2, 2) and mark it as visited.
 - Increment the island count to 2.



3. Finally, move to the bottom right corner starting at (3, 3):

- It's a '1', so we start another DFS.
- Mark (3, 3) and (3, 4) as visited.
- Increment the island count to 3.

After processing all cells, the final number of islands is **3**.

***SUDO CODE:-**

function numIslands(grid):

 if grid is empty:

 return 0

 rows = number of rows in grid

 cols = number of columns in grid

 define function dfs(i, j):

 if i is out of bounds or j is out of bounds or grid[i][j] is '0':

 return

 grid[i][j] = '0'

 dfs(i - 1, j)

 dfs(i + 1, j)

 dfs(i, j - 1)

 dfs(i, j + 1)

 island_count = 0

 for each cell (i, j) in grid:

 if grid[i][j] is '1':

 dfs(i, j)

 island_count += 1



```
return island_count
```

***PROGRAMM:-**

```
#include <stdio.h>
#include <stdlib.h>
#define ROWS 4
#define COLS 5
void dfs(char grid[ROWS][COLS], int i, int j, int rows, int cols) {
    if (i < 0 || i >= rows || j < 0 || j >= cols || grid[i][j] == '0') {
        return;
    }
    grid[i][j] = '0';
    dfs(grid, i - 1, j, rows, cols); // Up
    dfs(grid, i + 1, j, rows, cols); // Down
    dfs(grid, i, j - 1, rows, cols); // Left
    dfs(grid, i, j + 1, rows, cols); // Right
}
int numIslands(char grid[ROWS][COLS], int rows, int cols) {
    int island_count = 0;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (grid[i][j] == '1') {
                dfs(grid, i, j, rows, cols);
                island_count++;
            }
        }
    }
}
```



```
    }  
}  
    return island_count;  
}  
  
int main() {  
    char grid[ROWS][COLS] = {  
        {'1', '1', '0', '0', '0'},  
        {'1', '1', '0', '0', '0'},  
        {'0', '0', '1', '0', '0'},  
        {'0', '0', '0', '1', '1'}  
    };  
    printf("CH.keshavardhan 2203031050137\n");  
    printf("Number of islands: %d\n", numIslands(grid, ROWS, COLS));  
    return 0;  
}
```

***OUTPUT:-**

```
CH.keshavardhan 2203031050137  
Number of islands: 3  
  
=== Code Execution Successful ===
```




LEET CODE:-

```
class Solution(object):
    def numIslands(self, grid):
        """
        :type grid: List[List[str]]
        :rtype: int
        """
        if not grid:
            return 0
        rows = len(grid)
        cols = len(grid[0])
        def dfs(i, j):
            if i < 0 or i >= rows or j < 0 or j >= cols or grid[i][j] == '0':
                return
            grid[i][j] = '0'
            dfs(i - 1, j) # Up
            dfs(i + 1, j) # Down
            dfs(i, j - 1) # Left
            dfs(i, j + 1) # Right
        island_count = 0
        for i in range(rows):
            for j in range(cols):
                if grid[i][j] == '1':
                    dfs(i, j)
```



PARUL UNIVERSITY
FACULTY ENGINEERING & TECHNOLOGY
B.TECH CSE 3rd YEAR 5th SEMESTER
Design and Analysis of Algorithm
Laboratory (303105219)

```
island_count += 1  
return island_count
```

Input: grid = [
 ["1","1","0","0","0"],
 ["1","1","0","0","0"],
 ["0","0","1","0","0"],
 ["0","0","0","1","1"]
]

OUTPUT:-

```
CH.keshavardhan 2203031050137  
3  
  
Process finished with exit code 0
```



PRACTICAL-08

8. Given a grid of dimension $N \times M$ where each cell in the grid can have values 0, 1, or 2 which has the following meaning:

0: Empty cell

1: Cells have fresh

2: Cells have rotten oranges

We have to determine what is the minimum time required to rot all oranges. A rotten orange at index $[i, j]$ can rot other fresh oranges at indexes $[i-1, j]$, $[i+1, j]$, $[i, j-1]$, $[i, j+1]$ (up, down, left and right) in unit time.

BFS IMPLEMENTATION :- solve the problem of determining the minimum time required to rot all the oranges in the grid, we can approach it using **Breadth-First Search (BFS)**. The intuition behind this approach is that rotten oranges can affect their neighboring cells in all four directions (up, down, left, right) in one unit of time. This is similar to how BFS works, where we explore all nodes at the present "depth" (or distance) level before moving on to nodes at the next depth level.

***EXPLANATION:-**

1. Input Parsing & Initial Setup:

- We iterate over the grid to:
 - Add all rotten oranges (value 2) to the queue.
 - Count the number of fresh oranges (value 1).

2. BFS Traversal:

- We use a queue to process each rotten orange. For each rotten orange, we check its neighboring cells in the 4 possible directions (up, down, left, right).
- If a neighboring cell contains a fresh orange, we rot it (change its value to 2), reduce the fresh orange count, and add this cell to the queue for future processing.



3. Time Calculation:

- Each level of the BFS represents one unit of time since all the oranges in the queue rot their adjacent fresh oranges in parallel. After processing all oranges at one level, we increment the time counter.

4. Termination Condition:

- If after the BFS, there are no fresh oranges left, we return the time taken.
- If fresh oranges are still left unprocessed, it means it's impossible to rot all oranges, so we return -1.

Time Complexity:

- **$O(N \times M)$** : We visit each cell at most once, and we perform constant-time operations for each cell.
- **Space Complexity**: $O(N \times M)$, since in the worst case, the queue will contain all the oranges.

SUDO CODE:-

Input:

- Read the grid.

Initialize:

- Create a queue to store rotten oranges' positions.
- Count fresh oranges.

Queue Setup:

- For each cell in the grid:
 - If it's a rotten orange (value 2), add its position to the queue.
 - If it's a fresh orange (value 1), increment the fresh orange count.

Base Case:

- If no fresh oranges exist, return 0 (no time needed).



BFS Process (Rotting):

- Initialize minutes to 0.
- While the queue is not empty:
 - For each rotten orange in the queue:
 - For each of the four adjacent cells:
 - If the cell contains a fresh orange:
 - Rot the orange.
 - Add the new rotten orange to the queue.
 - Decrement the fresh orange count.
 - Increment the time (minutes) after each level of processing.

Final Check:

- If all fresh oranges are rotted, return minutes.
- Otherwise, return -1 (not all oranges can be rotted).

PROGRAMM:-

```
#include <stdio.h>
#include <stdbool.h>
#define MAX 1000
int directions[4][2] = { {-1, 0}, {1, 0}, {0, -1}, {0, 1} };
typedef struct {
    int x;
    int y;
} Point;
typedef struct {
```



```
Point points[MAX];
    int front, rear;
} Queue;
void initQueue(Queue* q) {
    q->front = 0;
    q->rear = 0;
}
bool isEmpty(Queue* q) {
    return q->front == q->rear;
}
void enqueue(Queue* q, int x, int y) {
    q->points[q->rear].x = x;
    q->points[q->rear].y = y;
    q->rear++;
}
Point dequeue(Queue* q) {
    Point p = q->points[q->front];
    q->front++;
    return p;
}
int minTimeToRotOranges(int grid[3][4], int rows, int cols) {
    Queue queue;
    initQueue(&queue);
    int fresh_oranges = 0;
```



```
for (int r = 0; r < rows; r++) {  
    for (int c = 0; c < cols; c++) {  
        if (grid[r][c] == 2) {  
            enqueue(&queue, r, c);  
        } else if (grid[r][c] == 1) {  
            fresh_oranges++;  
        }  
    }  
}  
  
if (fresh_oranges == 0) return 0;  
int minutes = 0;  
while (!isEmpty(&queue)) {  
    int num_rotten = queue.rear - queue.front;  
    bool rotted_any = false;  
    for (int i = 0; i < num_rotten; i++) {  
        Point p = dequeue(&queue);  
        int x = p.x;  
        int y = p.y;  
        for (int d = 0; d < 4; d++) {  
            int new_x = x + directions[d][0];  
            int new_y = y + directions[d][1];  
            if (new_x >= 0 && new_x < rows && new_y >= 0 && new_y < cols &&  
grid[new_x][new_y] == 1) {  
                grid[new_x][new_y] = 2;  
                enqueue(&queue, new_x, new_y);  
            }  
        }  
    }  
    minutes++;  
    if (isEmpty(&queue)) break;  
}
```



```
        fresh_oranges--;  
        rotted_any = true;  
    }  
}  
}  
if (rotted_any) {  
    minutes++;  
}  
}  
return fresh_oranges == 0 ? minutes : -1;  
}  
int main() {  
    int grid[3][4] = {  
        {2, 1, 0, 2},  
        {1, 1, 0, 1},  
        {0, 1, 2, 0}  
    };  
    int result = minTimeToRotOranges(grid, 3, 4);  
    printf("Minimum time to rot all oranges: %d\n", result);  
    printf("CH.keshavardhan 2203031050137\n");  
    return 0;  
}
```

OUTPUT:-

```
Minimum time to rot all oranges: 2  
CH.keshavardhan 2203031050137  
  
=== Code Execution Successful ===
```


PRACTICAL-09

9. Given two strings str1 and str2 and below operations that can be performed on str1. Find minimum number of edits (operations) required to convert 'str1' into 'str2'. Insert Remove Replace, All of the above operations are of equal cost.?

What is string:- A **string** is a sequence of characters, typically used to represent text. In programming, strings are a data type that holds a collection of characters, which can include letters, numbers, symbols, and spaces. Strings are usually enclosed in quotes—either single quotes (' ') or double quotes (" ").

Steps:

1. Base Case Initialization:

- If one of the strings is empty, the only way to transform it into the other is through insertions or deletions.
- $dp[i][0] = i$ for all i because it takes i deletions to convert $str1[0:i]$ to an empty string.
- $dp[0][j] = j$ for all j because it takes j insertions to convert an empty string to $str2[0:j]$.

2. Recurrence Relation:

- For each character in str1 and str2, if they are the same, then no additional operation is needed, i.e., $dp[i][j] = dp[i-1][j-1]$.
- If they are different, we need to choose the best operation (insert, remove, or replace):
 - **Insert:** Insert a character into str1 to match str2[j] $\rightarrow dp[i][j] = dp[i][j-1] + 1$
 - **Remove:** Remove a character from str1 to match str2[j] $\rightarrow dp[i][j] = dp[i-1][j] + 1$
 - **Replace:** Replace the character in str1 with the one in str2 $\rightarrow dp[i][j] = dp[i-1][j-1] + 1$
- The result is the minimum of the three operations:
 $dp[i][j] = \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1$



EXPLANATION:-

Base Case:

- The DP table is initialized such that converting an empty string to another string requires insertions, and vice versa.

Filling the Table:

- The DP table is filled based on character comparisons between str1 and str2, and we choose the minimum operation required at each step.

Result:

- The final answer is found in the bottom-right cell of the table, which contains the minimum number of operations needed to convert str1 into str2.

SUDO CODE:-

Function minEditDistance(str1, str2):

 m ← length of str1

 n ← length of str2

 dp ← 2D array of size (m+1) x (n+1)

 For i from 0 to m:

 For j from 0 to n:

 If i == 0:

 dp[i][j] ← j

 Else If j == 0:

 dp[i][j] ← i

 Else If str1[i-1] == str2[j-1]:

 dp[i][j] ← dp[i-1][j-1] # Characters match, no operation needed

 Else:



```
        dp[i][j] ← 1 + min(  
            dp[i-1][j],  
            dp[i][j-1],  
            dp[i-1][j-1]  
        )  
    Return dp[m][n]
```

PROGRAMM:-

```
#include <stdio.h>  
#include <string.h>  
#include <stdlib.h>  
int min(int x, int y, int z) {  
    if (x <= y && x <= z)  
        return x;  
    if (y <= x && y <= z)  
        return y;  
    return z;  
}  
int minEditDistance(char* str1, char* str2) {  
    int m = strlen(str1);  
    int n = strlen(str2);  
    int dp[m + 1][n + 1];  
    for (int i = 0; i <= m; i++) {  
        for (int j = 0; j <= n; j++) {
```

```
if (i == 0)
    dp[i][j] = j;
else if (j == 0)
    dp[i][j] = i;
else if (str1[i - 1] == str2[j - 1])
    dp[i][j] = dp[i - 1][j - 1];
else
    dp[i][j] = 1 + min(dp[i - 1][j], // Remove
                       dp[i][j - 1], // Insert
                       dp[i - 1][j - 1] // Replace
    );
}
}
return dp[m][n];
}

int main() {
    char str1[] = "sunday";
    char str2[] = "saturday";
    printf("Minimum Edit Distance: %d\n", minEditDistance(str1, str2));
    printf("CH.keshavardhan 2203031050137\n");
    return 0;
}
```

OUTPUT:-

```
Minimum Edit Distance: 3
CH.keshavardhan 2203031050137

=== Code Execution Successful ===
```



PRACTICAL-10

10. Minimum Path Sum” says that given a $n \times m$ grid consisting of non-negative integers and we need to find a path from top left to bottom right, which minimizes the sum of all numbers along the path?

Problem Solve Points:-

- **Input:** A 2D grid $grid[n][m]$ of size $n \times m$ where each cell contains a non-negative integer.
- **Output:** The minimum sum of the path from the top-left (0,0) to the bottom-right ($n-1$, $m-1$) corner.
- **Constraints:**
 - You can only move **right** ($\rightarrow$) or **down** ($\downarrow$).

EXPLANATION:-

- **First row:** We can only move right, so each cell's value is the sum of the current cell's value and the previous cell.
- **First column:** We can only move down, so each cell's value is the sum of the current cell and the one above it.
- **Rest of the grid:** For each cell, calculate the minimum of the values from the top or left and add the current cell's value.

Time Complexity:

- **$O(n * m)$:** We iterate over all the cells in the grid once to fill the DP table.

Space Complexity:

- **$O(n * m)$:** We use a separate DP table of size $n \times m$.



SUDO CODE:-

FUNCTION minPathSum(grid)

 n = number of rows in grid

 m = number of columns in grid

 Initialize dp as a 2D array of size n x m

 dp[0][0] = grid[0][0]

 FOR j FROM 1 TO m - 1 DO

 dp[0][j] = dp[0][j - 1] + grid[0][j]

 END FOR

 FOR i FROM 1 TO n - 1 DO

 dp[i][0] = dp[i - 1][0] + grid[i][0]

 END FOR

 FOR i FROM 1 TO n - 1 DO

 FOR j FROM 1 TO m - 1 DO

 dp[i][j] = grid[i][j] + MIN(dp[i - 1][j], dp[i][j - 1])

 END FOR

 END FOR

 RETURN dp[n - 1][m - 1]

END FUNCTION

Example usage:

PRINT minPathSum([

 [1, 3, 1],

 [1, 5, 1],

 [4, 2, 1]])



PRINT "CH.keshavardhan 2203031050137"

PROGRAM:-

```
#include <stdio.h>
#include <limits.h>
int minPathSum(int** grid, int m, int n) {
    int dp[m][n];
    dp[0][0] = grid[0][0];
    for (int j = 1; j < n; j++) {
        dp[0][j] = dp[0][j - 1] + grid[0][j];
    }
    for (int i = 1; i < m; i++) {
        dp[i][0] = dp[i - 1][0] + grid[i][0];
    }
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = grid[i][j] + (dp[i - 1][j] < dp[i][j - 1] ? dp[i - 1][j] : dp[i][j - 1]);
        }
    }
    return dp[m - 1][n - 1];
}
int main() {
    int m = 3, n = 3;
    int* grid[m];
    for (int i = 0; i < m; i++) {
```



```
grid[i] = (int*)malloc(n * sizeof(int));  
}  
grid[0][0] = 1; grid[0][1] = 3; grid[0][2] = 1;  
grid[1][0] = 1; grid[1][1] = 5; grid[1][2] = 1;  
grid[2][0] = 4; grid[2][1] = 2; grid[2][2] = 1;  
int result = minPathSum(grid, m, n);  
printf("Minimum Path Sum: %d\n", result);  
for (int i = 0; i < m; i++) {  
    free(grid[i]);  
}  
return 0;  
}
```

OUTPUT:-

```
7  
CH.keshavardhan 2203031050137  
  
Process finished with exit code 0
```


PRACTICAL-11

11. Given string num representing a non-negative integer num, and an integer k, return the smallest possible integer after removing k digits from num?

Problem Solve Points:-

1. Input:

- A string num representing a non-negative integer.
- An integer k representing the number of digits to remove.

2. Goal:

- Remove k digits from the number to form the smallest possible integer.

EXPLANATION:-

1. Stack:

- Use a stack to store the digits of the resulting number.
- As you iterate over each digit, you remove digits from the stack if the current digit is smaller than the last digit in the stack. This ensures that the smaller digits are kept at the beginning of the number to form a smaller number.

2. Greedy Removal:

- Start by iterating through the digits of the number. For each digit:
 - Remove the previous digits in the stack if they are larger than the current digit and if you still have digits left to remove (i.e., $k > 0$).
 - Push the current digit to the stack.

3. Remove Excess Digits:

- If there are still digits to remove after processing all the digits (i.e., $k > 0$), remove them from the end of the stack.

4. Edge Case:

- After building the stack, remove any leading zeros from the result.
- If the resulting number is empty after removing digits, return "0".



SUDO:-

FUNCTION removeKdigits(num, k)

 stack = EMPTY LIST

 FOR each digit in num:

 WHILE k > 0 AND stack is not empty AND stack[-1] > digit:

 POP the last element from the stack

 DECREMENT k by 1

 APPEND digit to stack

 WHILE k > 0:

 POP the last element from the stack

 DECREMENT k by 1

 result = JOIN elements of stack as a string

 REMOVE leading zeros from result

 IF result is empty:

 RETURN "0"

 ELSE:

 RETURN result

END FUNCTION

Example usage:

num = "1432219"

k = 3

PRINT removeKdigits(num, k) # Expected output: "1219"

PRINT "CH.keshavardhan 2203031050137"



PROGRAM:-

```
#include <stdio.h>
#include <string.h>

void removeKdigits(char* num, int k) {
    int len = strlen(num);
    char stack[len]; // This stack will hold the result
    int top = 0; // Points to the top of the stack
    for (int i = 0; i < len; i++) {
        char digit = num[i];
        while (k > 0 && top > 0 && stack[top - 1] > digit) {
            top--; // Pop the stack
            k--;
        }
        stack[top++] = digit;
    }
    while (k > 0 && top > 0) {
        top--; // Pop from the end
        k--;
    }
    int start = 0;
    while (start < top && stack[start] == '0') {
        start++;
    }
    if (start == top) {
```



```
        printf("0\n");
    } else {
        for (int i = start; i < top; i++) {
            printf("%c", stack[i]);
        }
        printf("\n");
    }
}

int main() {
    char num[] = "1432219";
    int k = 3;
    removeKdigits(num, k); // Expected output: "1219"
    printf("CH.keshavardhan 2203031050137\n");
    return 0;
}
```

OUTPUT:-

```
1219
CH.keshavardhan 2203031050137

Process finished with exit code 0
```

PRACTICAL-12

12. There is a robot on an $m \times n$ grid. The robot is initially located at the top-left corner (i.e., $\text{grid}[0][0]$). The robot tries to move to the bottom-right corner (i.e., $\text{grid}[m - 1][n - 1]$). The robot can only move either down or right at any point in time. Given the two integers m and n , return the number of possible unique paths that the robot can take to reach the bottom-right corner?

Problem Solve Points:-

- **$m \times n$ grid:**
 - m is the number of rows.
 - n is the number of columns.
- **Robot's movement:**
 - It can move **right** or **down** at any point in time.
- **Goal:**
 - Return the number of unique paths from the top-left corner to the bottom-right corner.

EXPLANATION:-

1. At any point, the robot can only move **right** or **down**. This means, to reach the destination, the robot must make exactly:
 - **$m-1$ down moves** (since there are m rows).
 - **$n-1$ right moves** (since there are n columns).
2. **Dynamic Programming Approach:**
 - We can break the problem into subproblems where each cell contains the number of ways to reach that cell from the top-left corner.
 - If the robot is at cell (i, j) , the number of ways to get there is the sum of:
 - The number of ways to get to the cell directly **above** it $(i-1, j)$, and
 - The number of ways to get to the cell directly **left** of it $(i, j-1)$.



3. Base Case:

If the robot is at the first row ($i = 0$), the only way to move is to go **right**. So, there's only 1 way to reach any cell in the first row.

Similarly, if the robot is at the first column ($j = 0$), the only way to move is to go **down**. So, there's only 1 way to reach any cell in the first column.

SUDO CODE:-

FUNCTION removeKdigits(num, k)

 stack = EMPTY LIST

 FOR each digit IN num:

 WHILE $k > 0$ AND stack is not empty AND $\text{stack}[\text{top}] > \text{digit}$:

 POP the last element from stack

 DECREMENT k by 1

 APPEND digit TO stack

 WHILE $k > 0$:

 POP the last element from stack

 DECREMENT k by 1

 result = JOIN stack elements as a string

 result = REMOVE leading zeros from result

 IF result is empty:

 RETURN "0"

 ELSE:

 RETURN result

END FUNCTION



Example usage:

```
num = "1432219"
```

```
k = 3
```

```
PRINT removeKdigits(num, k) # Expected output: "1219"
```

```
PRINT "CH.keshavardhan 2203031050137"
```

PROGRAM:-

```
#include <stdio.h>
```

```
int uniquePaths(int m, int n) {
```

```
    int dp[m][n];
```

```
    for (int i = 0; i < m; i++) {
```

```
        dp[i][0] = 1;
```

```
    }
```

```
    for (int j = 0; j < n; j++) {
```

```
        dp[0][j] = 1;
```

```
    }
```

```
    for (int i = 1; i < m; i++) {
```

```
        for (int j = 1; j < n; j++) {
```

```
            dp[i][j] = dp[i-1][j] + dp[i][j-1];
```

```
        }
```

```
    }
```

```
    return dp[m-1][n-1];
```

```
}
```

```
int main() {
```

```
    int m = 3;
```



PARUL UNIVERSITY
FACULTY ENGINEERING & TECHNOLOGY
B.TECH CSE 3rd YEAR 5th SEMESTER
Design and Analysis of Algorithm
Laboratory (303105219)

```
int n = 7;  
printf("%d\n", uniquePaths(m, n));  
return 0;  
}
```

OUTPUT:-

```
28  
CH.keshavardhan 2203031050137  
  
Process finished with exit code 0
```